0039. Combination Sum

edium | Category: Backtracking | LeetCode: https://leetcode.com/problems/comb

Problem

- Distinct candidates, each usable unlimited times. Return all unique combinations summing to target.
- Example: [2,3,6,7], target 7 → [[2,2,3],[7]].

Intuition

- To avoid permutations of the same combination, only pick candidates at index >= the last one picked.
- Reusing an element = recursing with the same start index.

Brute Force

- Generate all sequences and dedupe by sorting: exponential with heavy duplication.

Optimal Approach

- Sort candidates (enables early break).
- Dfs(start, remaining): if remaining == 0 record.
- For i from start: if c[i] > remaining break; add, dfs(i, remaining - c[i]), remove.

Worked Example

```
[2,3,6,7], target 7.

2 -> 2 -> 2 -> (rem 1) dead
          -> 3 -> rem 0 record [2,2,3]
     3 -> rem 2: 3 > 2 break
3 -> 3 -> rem 1 dead
6 -> rem 1 dead
7 -> rem 0 record [7]
```

Java Solution

```java
public class CombinationSum {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        int[] c = candidates.clone();
        Arrays.sort(c);
        List<List<Integer>> res = new ArrayList<>();
        dfs(c, 0, target, new ArrayList<>(), res);
        return res;
    }

    private void dfs(int[] c, int start, int rem, List<Integer> path, List<List<Integer>> res) {
        if (rem == 0) { res.add(new ArrayList<>(path)); return; }
        for (int i = start; i < c.length && c[i] <= rem; i++) {
            path.add(c[i]);
            dfs(c, i, rem - c[i], path, res);
            path.remove(path.size() - 1);
        }
    }
}
```

Complexity

- Time: O(N^(T/M + 1)) loosely (T target, M min candidate). Space: O(T/M) depth.

Pitfalls

- Passing i + 1 instead of i forbids reuse (that is problem 40).